

Chicago Accessibility Analysis - Project Walkthrough

Chicago Transit & Service Accessibility Analysis

Complete Project Walkthrough — Code, Explanations & Resume Guide

How to Use This Document

1. Project Goal & Data Sources
 2. Database Setup (SQL)
 3. Python Data Loading Scripts
 6. Deployment: Cloud Database + Live Dashboard
 8. Final Summary of Key Findings
- Appendix: Full Project File Reference

Chicago Transit & Service Accessibility Analysis

Complete Project Walkthrough — Code, Explanations & Resume Guide

Author: Ofobutu Abiodun Emmanuel (Abbeycity) **GitHub:** github.com/Abiodun360of/chicago-accessibility-analysis **Live Dashboard:** chicago-accessibility-analysis-idfqytpujbiletwrmbxxh.streamlit.app

How to Use This Document

This is your complete reference for the entire project — every piece of code you wrote, in order, with a plain-English explanation of what it does and why. If you forget what a script does or need to explain the project to someone else (an interviewer, a client), come back here.

The document follows the exact order you built the project in:

1. Project Goal & Data Sources
2. Database Setup (SQL)
3. Python Data Loading Scripts (4 scripts)
4. SQL Analysis Queries
5. R Statistical Analysis & Machine Learning
6. Deployment (Supabase + Streamlit)
7. **How to resume work in RStudio** (practical restart guide)

1. Project Goal & Data Sources

The Goal

Build a system that scores Chicago neighborhoods (Census tracts) by how well they are served by public transit, schools, hospitals, and grocery stores — identifying underserved “accessibility deserts” using real, open-source geospatial data. This demonstrates database management, spatial SQL, Python, R, and machine learning skills for GIS Analyst / geospatial data roles.

Why a Spatial Database (Not Just a Spreadsheet)?

The core objects in this project — bus stops, roads, neighborhood boundaries, schools — all exist at specific locations in physical space. Answering questions like “is this point inside this boundary?” or “how far is this point from that point?” requires spatial operations that a normal spreadsheet cannot do. PostGIS is an extension to PostgreSQL that adds this spatial capability directly into the database.

Data Sources

Dataset	Source	Records	Format in real world
Transit stops	CTA (Chicago Transit Authority) GTFS feed	11,177	Point locations
Road network	OpenStreetMap (via OSMnx)	77,448	Line segments
Population zones	US Census Bureau (TIGER boundaries + ACS demographic survey)	1,331 Cook County tracts → 870 filtered to actual Chicago	Polygon boundaries
Points of interest (schools, hospitals, groceries)	OpenStreetMap	1,358	Points (some converted from polygons)

Why these sources: all four are free, public, well-documented industry-standard sources, and are accessible programmatically (via code, not manual downloads) — which is exactly how real GIS/data analyst jobs work with data.

2. Database Setup (SQL)

2.1 Creating the Four Core Tables

Each table's geometry type matches what it represents in the real world: points for single locations, lines for roads, polygons for boundaries.

```
-- Create table for CTA transit stops (point locations)
CREATE TABLE transit_stops (
  stop_id SERIAL PRIMARY KEY,
  stop_name VARCHAR(255),
  route_type VARCHAR(50),
  geom GEOMETRY(Point, 4326)
);

-- Create table for Chicago road network (line segments)
CREATE TABLE roads (
  road_id SERIAL PRIMARY KEY,
  road_name VARCHAR(255),
  road_type VARCHAR(50),
  geom GEOMETRY(LineString, 4326)
);

-- Create table for Census tracts with population/income data
CREATE TABLE population_zones (
  zone_id SERIAL PRIMARY KEY,
  tract_name VARCHAR(255),
  population INTEGER,
  median_income NUMERIC,
  geom GEOMETRY(MultiPolygon, 4326)
);

-- Create table for points of interest (schools, hospitals, groceries)
CREATE TABLE poi (
  poi_id SERIAL PRIMARY KEY,
  poi_name VARCHAR(255),
  poi_type VARCHAR(50),
  geom GEOMETRY(Point, 4326)
);
```

Explanation of each piece:

- `CREATE TABLE tablename (...)` — makes a new table with the listed columns.
- `SERIAL PRIMARY KEY` — an auto-incrementing unique ID column (1, 2, 3...) that never has to be set manually; guarantees every row is uniquely identifiable.
- `VARCHAR(255)` — a text column with a maximum length.
- `GEOMETRY(Point, 4326)` — the special PostGIS column type. `Point` (or `LineString`, `MultiPolygon`) defines what shape this column stores. `4326` is the SRID (Spatial Reference System Identifier) — this is the standard code for regular latitude/longitude coordinates (WGS84), used by GPS, Google Maps, and virtually all web mapping.
- `MultiPolygon` (rather than plain `Polygon`) was used for tract boundaries because some real tracts are made of multiple disconnected pieces (e.g., split by a river or highway); `MultiPolygon` safely handles both single- and multi-piece shapes.

2.2 Spatial Indexes

Without an index, PostGIS must check every single row to answer a proximity question — extremely slow at scale (e.g., 77,448 road rows). A spatial index pre-organizes location data so PostGIS can quickly narrow down to a relevant subset instead of scanning everything.

```
-- Create spatial indexes for fast proximity/distance queries on all geometry columns
CREATE INDEX idx_transit_stops_geom ON transit_stops USING GIST (geom);
CREATE INDEX idx_roads_geom ON roads USING GIST (geom);
CREATE INDEX idx_population_zones_geom ON population_zones USING GIST (geom);
CREATE INDEX idx_poi_geom ON poi USING GIST (geom);
```

`GIST` is the specific indexing algorithm PostGIS uses for spatial data — the standard choice, no deeper detail needed beyond “it’s the correct index type for geometry columns.”

3. Python Data Loading Scripts

All four scripts follow a shared pattern: **connect to the database → pull data from a source → clean/reshape it → write it into the matching PostGIS table.**

3.1 load_stops.py — CTA Transit Stops

```
import pandas as pd
import psycopg2

# Connect to the database
conn = psycopg2.connect(
    dbname="chicago_transit",
    user="postgres",
    password="postgres",
    host="localhost",
    port="5433"
)
cur = conn.cursor()

# Load the CTA GTFS stops file
stops_df = pd.read_csv(r"C:\Users\user\Desktop\chicago\stops.txt")

# Insert each stop into the database, one row at a time
for index, row in stops_df.iterrows():
    cur.execute("""
        INSERT INTO transit_stops (stop_name, geom)
        VALUES (%s, ST_SetSRID(ST_MakePoint(%s, %s), 4326))
        """, (row['stop_name'], row['stop_lon'], row['stop_lat']))

conn.commit()
print(f"Inserted {len(stops_df)} stops successfully!")
```

Explanation:

- `import pandas as pd / import psycopg2` — load external tools: `pandas` for working with table-like data, `psycopg2` for talking to PostgreSQL from Python.
- `psycopg2.connect(...)` — logs Python into the database, same credentials as pgAdmin. `cur = conn.cursor()` creates a “pen” for sending SQL commands and receiving results.
- `pd.read_csv(...)` — loads the CTA’s `stops.txt` file (a spreadsheet, despite the `.txt` extension) into a table called `stops_df`.
- `for index, row in stops_df.iterrows():` — loops through the spreadsheet one row (one bus stop) at a time.
- `INSERT INTO transit_stops (stop_name, geom) VALUES (%s, ST_SetSRID(ST_MakePoint(%s, %s), 4326))` — the SQL command that adds a new row. `ST_MakePoint(longitude, latitude)` builds an actual map point from two numbers (note: longitude first, then latitude — a common source of bugs if reversed). `ST_SetSRID(..., 4326)` tags that point with the standard coordinate system so PostGIS knows how to interpret it.
- `%s` placeholders + the tuple `(row['stop_name'], row['stop_lon'], row['stop_lat'])` — parameterized query values, inserted safely (protects against SQL injection, standard practice).
- `conn.commit()` — permanently saves all the inserts. Nothing is actually written to the database until this line runs.

Result: 11,177 rows inserted into `transit_stops`.

3.2 load_roads.py — Chicago Road Network

```

import osmnx as ox

# Download the road network for Chicago from OpenStreetMap
G = ox.graph_from_place("Chicago, Illinois, USA", network_type="drive")

print("Number of nodes (intersections):", len(G.nodes))
print("Number of edges (road segments):", len(G.edges))

# Convert the graph edges (road segments) into a table
edges = ox.graph_to_gdfs(G, nodes=False, edges=True)

import geopandas as gpd
from sqlalchemy import create_engine

# Reset the index so u, v, key become normal columns instead of a multi-index
edges = edges.reset_index()

# Keep only the columns we need, and rename them to match our table
roads_gdf = edges[['name', 'highway', 'geometry']].copy()
roads_gdf = roads_gdf.rename(columns={'name': 'road_name', 'highway': 'road_type'})

# Some road_name/road_type values are lists (a road with two names) - convert to text
roads_gdf['road_name'] = roads_gdf['road_name'].astype(str)
roads_gdf['road_type'] = roads_gdf['road_type'].astype(str)

# Rename the geometry column so it matches our table's 'geom' column
roads_gdf = roads_gdf.rename_geometry('geom')

# Set the coordinate reference system explicitly to WGS84 (lat/long)
roads_gdf = roads_gdf.set_crs("EPSG:4326", allow_override=True)

# Write to PostGIS, appending to our existing empty 'roads' table
engine = create_engine("postgresql://postgres:postgres@localhost:5433/chicago_transit")
roads_gdf.to_postgis("roads", engine, if_exists="append", index=False)

print(f"Inserted {len(roads_gdf)} road segments successfully!")

```

Explanation — the graph concept (nodes and edges):

A road network is naturally a *graph*: **nodes** are intersections (or dead-ends), and **edges** are the individual road segments connecting them. `ox.graph_from_place(...)` downloads Chicago's entire drivable street network from OpenStreetMap in this graph structure. `ox.graph_to_gdfs(G, nodes=False, edges=True)` converts just the edges (roads) into a spreadsheet-like table — we don't need the intersection points themselves for this table.

Explanation — cleaning steps:

- `.reset_index()` — resets messy automatic row labels to simple numbering.
- Selecting `['name', 'highway', 'geometry']` and renaming to `road_name`, `road_type` — keeps only the three columns we need and matches our database's column names.
- `.astype(str)` — OSM sometimes stores a road's name/type as a list (e.g., a road with two official names); converting to plain text avoids errors when inserting into a `VARCHAR` column.
- `.rename_geometry('geom')` — **this was the fix for a real bug we hit**. Geopandas' default geometry column name is `geometry`, but our database table's column is `geom`. Without this rename, `.to_postgis()` fails with an SRID error because it can't find a `geometry` column in a table that only has `geom`. Lesson: automated tools assume default names, and mismatches like this are a common, learnable class of bug.
- `.set_crs("EPSG:4326", ...)` — confirms the standard coordinate system, same idea as `ST_SetSRID` in raw SQL.
- `create_engine(...)` — SQLAlchemy's connection manager, needed specifically for the `.to_postgis()` bulk-write shortcut (different from `psycopg2`'s row-by-row approach used in `load_stops.py`).
- `.to_postgis("roads", engine, if_exists="append", index=False)` — writes the *entire* table into PostGIS in one efficient operation, rather than looping row by row.

Result: 77,448 rows inserted into `roads`.

3.3 `load_population.py` — Census Tracts, Population & Income

```

from pygris import tracts
import requests
import pandas as pd
from sqlalchemy import create_engine

api_key = "e2751cb55d5b31578ef632b0df6c7515d06139cb"

# Step 1: Get tract boundaries (shapes) for Cook County, Illinois
il_tracts = tracts(state="IL", county="Cook", cb=True, year=2022)
print("Tract boundaries pulled:", len(il_tracts))

# Step 2: Pull population (B01003_001E) and median household income (B19013_001E)
url = f"https://api.census.gov/data/2022/acs/acs5?get=NAME,B01003_001E,B19013_001E&for=tract:*&in=state:17+county:031&key={api_key}"
response = requests.get(url)
data = response.json()

census_df = pd.DataFrame(data[1:], columns=data[0])
print("Census rows pulled:", len(census_df))

# Step 3: Join census data to tract boundaries using GEOID (an exact numeric match,
# unlike tract names which can have inconsistent formatting)
census_df['GEOID'] = census_df['state'] + census_df['county'] + census_df['tract']
merged = il_tracts.merge(census_df, on='GEOID', how='inner')
print("Merged rows:", len(merged))

# Step 4: Prepare data for PostGIS
pop_gdf = merged[['NAME_x', 'B01003_001E', 'B19013_001E', 'geometry']].copy()
pop_gdf = pop_gdf.rename(columns={
    'NAME_x': 'tract_name',
    'B01003_001E': 'population',
    'B19013_001E': 'median_income'
})

# Convert to numbers; Census uses negative codes (e.g. -666666666) for suppressed/
# missing data, so we treat any negative income as missing rather than a real value
pop_gdf['population'] = pd.to_numeric(pop_gdf['population'], errors='coerce')
pop_gdf['median_income'] = pd.to_numeric(pop_gdf['median_income'], errors='coerce')
pop_gdf.loc[pop_gdf['median_income'] < 0, 'median_income'] = None

pop_gdf = pop_gdf.rename_geometry('geom')
pop_gdf = pop_gdf.set_crs("EPSG:4326", allow_override=True)

engine = create_engine("postgres://postgres:postgres@localhost:5433/chicago_transit")
pop_gdf.to_postgis("population_zones", engine, if_exists="append", index=False)

print(f"Inserted {len(pop_gdf)} population zones successfully!")

```

Explanation:

- `pygris` — a Python package built specifically for pulling US Census TIGER geographic boundaries.
- `tracts(state="IL", county="Cook", cb=True, year=2022)` — pulls the 1,331 Census tract *shapes* for Cook County (`cb=True` = simplified “cartographic boundary” version, good enough for mapping/analysis and smaller/faster than the full-detail legal version).
- The Census API URL — `B01003_001E` and `B19013_001E` are official Census variable codes for total population and median household income respectively. `state:17+county:031` restricts the request to Illinois, Cook County only (otherwise it would return the whole USA).
- `response.json()` — converts the raw API response text into Python data; `pd.DataFrame(data[1:], columns=data[0])` treats the first row as column headers and everything else as data.
- **The join (Step 3):** two separate tables (shapes, and population/income numbers) don’t know how to connect to each other by default. `GEOID` is the Census Bureau’s official unique tract identifier, built by concatenating state + county + tract codes — using it instead of tract *names* avoids mismatches from inconsistent text formatting between the two sources. `.merge(..., on='GEOID', how='inner')` is Python’s equivalent of a SQL `JOIN`.
- **Missing data handling:** the Census Bureau uses specific negative numbers (like -666666666) as a code meaning “data suppressed/unavailable” (often for privacy in small tracts). Treating any negative income as missing (`None`) prevents nonsensical negative-income values from corrupting later statistics.
- The rest (`rename_geometry`, `set_crs`, `to_postgis`) follows the same pattern as `load_roads.py`.

Result: 1,331 rows inserted into `population_zones` (later filtered down to 870 actual Chicago tracts — see Section 4.3).

3.4 `load_poi.py` — Schools, Hospitals, Groceries

```

import osmnx as ox

# Define the OSM tags we want to search for
tags = {
    'amenity': ['school', 'hospital'],
    'shop': ['supermarket', 'grocery']
}

# Pull matching points of interest for Chicago
poi_gdf = ox.features_from_place("Chicago, Illinois, USA", tags)

poi_gdf = poi_gdf.reset_index()

# Convert all geometries to points: if it's already a point, keep it;
# if it's a polygon (e.g. a hospital building outline), use its centroid instead
poi_gdf['geom'] = poi_gdf.geometry.apply(
    lambda geom: geom if geom.geom_type == 'Point' else geom.centroid
)

# Combine the 'amenity' and 'shop' tag columns into one clean 'poi_type' column
poi_gdf['poi_type'] = poi_gdf['amenity'].fillna(poi_gdf['shop'])

poi_final = poi_gdf[['name', 'poi_type', 'geom']].copy()
poi_final = poi_final.rename(columns={'name': 'poi_name'})

import geopandas as gpd
poi_final = gpd.GeoDataFrame(poi_final, geometry='geom')
poi_final = poi_final.set_crs("EPSG:4326", allow_override=True)

from sqlalchemy import create_engine
engine = create_engine("postgresql://postgres:postgres@localhost:5433/chicago_transit")
poi_final.to_postgis("poi", engine, if_exists="append", index=False)

print(f"Inserted {len(poi_final)} POIs successfully!")

```

Explanation:

- `tags = {...}` — OpenStreetMap labels real-world places with key-value tags (e.g. `amenity=school`). This dictionary specifies exactly which tag combinations to search for.
- `ox.features_from_place(...)` — searches OpenStreetMap for every feature matching those tags within Chicago's boundary. Returned 1,358 results: a mix of 350 points and 1,008 polygons (988 Polygon + 20 MultiPolygon).
- **The centroid conversion (the key new concept in this file):** our `poi` table only stores `Point` geometry, but many real-world places (like a hospital) are mapped in OSM as building outlines (polygons), not single points. `geom.centroid` calculates the mathematical center point of a shape — like balancing a cutout of the shape on a pin, that balance point is the centroid. `geom.apply(lambda geom: geom if geom.geom_type == 'Point' else geom.centroid)` checks each row: if it's already a point, keep it; otherwise, replace it with its centroid. This loses the exact building outline but gains a single usable location — exactly what's needed for proximity analysis.
- `.fillna()` — some rows have their type in the `amenity` column (schools, hospitals), others in `shop` (supermarkets, groceries), with the other column empty for each row. `.fillna()` merges these into one clean `poi_type` column by using whichever value isn't empty.
- Rest of the script follows the same load pattern as the previous files.

Result: 1,358 rows inserted into `poi` (breakdown: 1,042 schools, 271 supermarkets, 40 hospitals, 4 grocery, 1 stray restaurant tag).

3.5 `get_chicago_boundary.py` — Fixing a Real Data Scope Bug

While analyzing the data (Section 4.3), we discovered `population_zones` incorrectly included Cook County suburbs (Cicero, Berwyn, Evanston, etc.), not just Chicago proper. This script pulled Chicago's actual official city boundary to fix that.

```

from pygris import places
import geopandas as gpd

# Pull all incorporated places in Illinois, then filter to just Chicago
il_places = places(state="IL", cb=True, year=2022)
chicago_boundary = il_places[il_places['NAME'] == 'Chicago']

print(chicago_boundary[['NAME', 'geometry']])
print("Found:", len(chicago_boundary), "matching boundary")

from sqlalchemy import create_engine

chicago_boundary = chicago_boundary.rename_geometry('geom')
chicago_boundary = chicago_boundary.set_crs("EPSG:4326", allow_override=True)

engine = create_engine("postgresql://postgres:postgres@localhost:5433/chicago_transit")
chicago_boundary[['NAME', 'geom']].to_postgis("chicago_boundary", engine, if_exists="replace", index=False)

print("Chicago boundary loaded into PostGIS!")

```

Explanation: `places(...)` from `pygris` pulls every incorporated city/town boundary in Illinois; filtering `NAME == 'Chicago'` isolates exactly one row — Chicago's official city limits polygon. `if_exists="replace"` (not `"append"`) is used here because this is a single-row reference table being created fresh, not something built up incrementally like the other tables. ## 4. SQL Analysis Queries

4.1 First Attempt: Stops Near Each Zone

```

SELECT
    p.zone_id,
    p.tract_name,
    p.population,
    COUNT(t.stop_id) AS nearby_stops
FROM population_zones p
LEFT JOIN transit_stops t
    ON ST_DWithin(p.geom, t.geom, 0.005)
GROUP BY p.zone_id, p.tract_name, p.population
ORDER BY nearby_stops DESC
LIMIT 10;

```

Explanation:

- `FROM population_zones p` — start from the zones table, aliased as `p` for brevity.
- `LEFT JOIN transit_stops t ON ST_DWithin(p.geom, t.geom, 0.005)` — this is a *spatial* join: instead of matching rows by a shared ID, we match a zone to a stop whenever `ST_DWithin` says they're within 0.005 (degrees) of each other. `ST_DWithin(geom_a, geom_b, distance)` checks proximity within a given distance.
- **Why `LEFT JOIN` and not plain `JOIN`:** a plain `JOIN` would silently drop any zone with zero matching stops. Since the entire point of this analysis is to find zones with *few or no* stops, we cannot afford to lose them — `LEFT JOIN` keeps every zone regardless of whether it found a match.
- `COUNT(t.stop_id) AS nearby_stops` — counts how many stops matched each zone.
- `GROUP BY p.zone_id, p.tract_name, p.population` — **required whenever aggregating** (like `COUNT`). Without this, the join produces one row per matching stop (a zone with 61 nearby stops would appear as 61 separate rows). `GROUP BY` collapses all of a zone's matching rows into a single summary row, which is what makes `COUNT()` meaningful per zone rather than across the whole result.
- `ORDER BY nearby_stops DESC LIMIT 10` — shows the 10 best-served zones first (used initially just to sanity-check the query worked).

4.2 Finding the Least-Served Zones

```

SELECT
    p.zone_id,
    p.tract_name,
    p.population,
    COUNT(t.stop_id) AS nearby_stops
FROM population_zones p
LEFT JOIN transit_stops t
    ON ST_DWithin(p.geom, t.geom, 0.005)
GROUP BY p.zone_id, p.tract_name, p.population
HAVING p.population > 500
ORDER BY nearby_stops ASC
LIMIT 10;

```


New piece: `HAVING p.population > 500` — filters *after* grouping/aggregation (unlike `WHERE`, which filters *before*). Without this, zero-population areas (parks, industrial land) would clutter the “worst-served” results with meaningless zero counts. `HAVING` restricts results to zones with real residents, making the output actually meaningful.

Problem discovered: several zones showed exactly 0 nearby stops. This turned out to be a real data-quality bug (see 4.3), not a true finding.

4.3 Diagnosing and Fixing the Cook County vs. Chicago Bug

Step 1 — verify a specific zero-result zone by finding its true nearest stop regardless of distance:

```
SELECT
  p.zone_id,
  p.tract_name,
  t.stop_name,
  ST_Distance(p.geom, t.geom) AS distance_degrees
FROM chicago_zones p, transit_stops t
WHERE p.zone_id = 1150
ORDER BY distance_degrees ASC
LIMIT 5;
```

This showed the actual nearest stop was ~0.0123 degrees away (roughly 1.4 km) — legitimately far, not a bug for that particular zone. But separately, checking zone coordinates revealed the deeper issue: many “zero-stop” zones were physically located in suburbs like Cicero and Berwyn, not Chicago itself — because `population_zones` was built from all of **Cook County**, while `transit_stops` only contains **CTA** (Chicago’s own transit agency) data. Suburbs use a different agency (Pace), which we never pulled.

Step 2 — confirm the scope mismatch:

```
SELECT COUNT(*)
FROM population_zones p, chicago_boundary c
WHERE ST_Intersects(p.geom, c.geom);
```

Result: only 870 of the 1,331 tracts were actually inside Chicago city limits — confirming the bug.

Step 3 — create a reusable view to permanently fix the scope:

```
CREATE VIEW chicago_zones AS
SELECT p.*
FROM population_zones p, chicago_boundary c
WHERE ST_Intersects(p.geom, c.geom);
```

Explanation:

- `ST_Intersects(a, b)` — checks whether two shapes actually touch or overlap at all (different from `ST_DWithin`, which checks distance between things that may not overlap). Used here to test “does this population zone’s shape overlap Chicago’s official boundary at all?”
- `FROM population_zones p, chicago_boundary c` — an older SQL style of writing a join (comma-separated tables), combined with the `WHERE` filter that acts as the join condition.
- `CREATE VIEW chicago_zones AS ...` — this **saves the query permanently under a name**. A view is not a stored copy of data; it’s a saved question that re-runs live every time it’s used. From this point on, `SELECT * FROM chicago_zones` always returns only the 870 real Chicago tracts, without having to repeat the `ST_Intersects` filter in every future query.

4.4 Fixing Distance Accuracy: The `::geography` Cast

The original `0.005` degree threshold was only a rough approximation, valid specifically at Chicago’s latitude. Degrees are not a consistent real-world distance everywhere on Earth (one degree of longitude covers more real ground near the equator than near the poles).

```

SELECT
  p.zone_id,
  p.tract_name,
  p.population,
  COUNT(t.stop_id) AS nearby_stops
FROM chicago_zones p
LEFT JOIN transit_stops t
  ON ST_DWithin(p.geom::geography, t.geom::geography, 500)
GROUP BY p.zone_id, p.tract_name, p.population
HAVING p.population > 500
ORDER BY nearby_stops ASC
LIMIT 10;

```

Explanation:

- `p.geom::geography` — the `::` operator casts a value to a different type for a calculation. Casting from `geometry` (flat, degree-based math) to `geography` tells PostGIS to calculate distance on the curved surface of the real Earth, in actual meters — accurate everywhere, not just at one specific latitude.
- `500` — now this number is unambiguous: exactly 500 real-world meters (a commonly used “walkable distance” benchmark), rather than an arbitrary degree guess.
- **Trade-off learned:** this version is more accurate but noticeably slower (a few seconds vs. instant), since casting to `geography` on every row partially bypasses the spatial index built for raw `geometry`. Worth knowing as a real, common accuracy-vs-speed trade-off in PostGIS.

4.5 Final Combined Multi-Category Accessibility Query

```

SELECT
  p.zone_id,
  p.tract_name,
  p.population,
  COUNT(DISTINCT t.stop_id) AS nearby_stops,
  COUNT(DISTINCT CASE WHEN poi.poi_type = 'school' THEN poi.poi_id END) AS nearby_schools,
  COUNT(DISTINCT CASE WHEN poi.poi_type = 'hospital' THEN poi.poi_id END) AS nearby_hospitals,
  COUNT(DISTINCT CASE WHEN poi.poi_type IN ('supermarket', 'grocery') THEN poi.poi_id END) AS nearby_groceries
FROM chicago_zones p
LEFT JOIN transit_stops t ON ST_DWithin(p.geom::geography, t.geom::geography, 500)
LEFT JOIN poi ON ST_DWithin(p.geom::geography, poi.geom::geography, 800)
GROUP BY p.zone_id, p.tract_name, p.population
ORDER BY p.zone_id
LIMIT 15;

```

Explanation — the two new concepts here:

- 1. Joining two tables to the same base table causes row multiplication.** When a zone matches, say, 10 stops AND 5 POIs, the join produces every possible stop-POI combination for that zone: $10 \times 5 = 50$ rows. Each individual stop now appears 5 times (once per matching POI), and vice versa.
- 2. `COUNT(DISTINCT ...)` fixes the multiplication problem.** Without `DISTINCT`, `COUNT(t.stop_id)` on that 50-row block would return 50 — wildly inflated. `COUNT(DISTINCT t.stop_id)` counts each *unique* stop ID only once, no matter how many duplicate rows it appears across, correcting the count back down to the true value (10).
- 3. `CASE WHEN ... THEN ... END` inside `COUNT` — counting subcategories in one query.** `CASE WHEN poi.poi_type = 'school' THEN poi.poi_id END` acts as an inline filter per row: “if this row’s POI is a school, return its ID; otherwise return nothing (NULL).” Wrapped in `COUNT(DISTINCT ...)`, this counts only the unique school IDs that passed the check — `COUNT()` automatically ignores NULLs, so non-schools are silently excluded. Repeating this pattern for hospitals and groceries lets one single query return a complete accessibility profile per zone, instead of running four separate queries.

Worked example (small numbers) illustrating why `DISTINCT` matters:

Zone A has 2 stops (Stop 1, Stop 2) and 3 POIs (School X, School Y, Hospital Z). The double join produces $2 \times 3 = 6$ rows — every stop-POI combination. `COUNT(stop_id)` without `DISTINCT` = 6 (wrong; true answer is 2). `COUNT(DISTINCT stop_id)` = 2 (correct). Same logic applies to the POI count.

4.6 Additional Queries Used During ML Feature Engineering (Section 5)

Zone area (for population density):

```

SELECT zone_id, ST_Area(geom::geography) AS area_sqm
FROM chicago_zones

```

`ST_Area(geom::geography)` calculates each zone's real-world area in square meters, using the same accurate `::geography` approach as the distance queries.

Distance from downtown Chicago:

```
SELECT
  zone_id,
  ST_Distance(
    geom::geography,
    ST_SetSRID(ST_MakePoint(-87.6298, 41.8781), 4326)::geography
  ) AS distance_from_downtown
FROM chicago_zones
```

`ST_MakePoint(-87.6298, 41.8781)` creates a single reference point at downtown Chicago's coordinates (The Loop).

`ST_Distance(...)` then measures each zone's real-world distance from that one fixed point, in meters. ## 5. R Statistical Analysis & Machine Learning

5.1 Connecting R to the Database

```
library(DBI)
library(RPostgres)

con <- dbConnect(
  RPostgres::Postgres(),
  dbname = "chicago_transit",
  host = "localhost",
  port = 5433,
  user = "postgres",
  password = "postgres"
)
```

Explanation: `library(...)` loads external R tools, same idea as Python's `import`. `dbConnect(...)` is R's equivalent of `psycopg2.connect()` — same database, same credentials, different language. `con` represents the open connection used for every query from here on.

Important operational note: every time RStudio is closed or restarted, `con` and every other variable disappear from memory. The full reconnect block must be re-run at the start of each new session (see Section 7).

5.2 Pulling the Combined Accessibility Dataset into R

```
query <- "
SELECT
  p.zone_id,
  p.tract_name,
  p.population,
  p.median_income,
  COUNT(DISTINCT t.stop_id) AS nearby_stops,
  COUNT(DISTINCT CASE WHEN poi.poi_type = 'school' THEN poi.poi_id END) AS nearby_schools,
  COUNT(DISTINCT CASE WHEN poi.poi_type = 'hospital' THEN poi.poi_id END) AS nearby_hospitals,
  COUNT(DISTINCT CASE WHEN poi.poi_type IN ('supermarket','grocery') THEN poi.poi_id END) AS nearby_groceries
FROM chicago_zones p
LEFT JOIN transit_stops t ON ST_DWithin(p.geom::geography, t.geom::geography, 500)
LEFT JOIN poi ON ST_DWithin(p.geom::geography, poi.geom::geography, 800)
GROUP BY p.zone_id, p.tract_name, p.population, p.median_income
ORDER BY p.zone_id
"

accessibility_data <- dbGetQuery(con, query)
```

Explanation: the SQL text (identical logic to Section 4.5, with `median_income` added) is stored as a string in R.

`dbGetQuery(con, query)` sends it through the connection, runs it, and returns the result as an R data table — this is R's equivalent of fetching results after `cur.execute()` in Python. This query takes roughly 30–40 seconds due to the double spatial join across ~870 zones.

Save immediately to avoid re-running this slow query every session:

```
saveRDS(accessibility_data, "accessibility_data.rds")

# In any future session, reload instantly instead of re-querying:
accessibility_data <- readRDS("accessibility_data.rds")
```

5.3 Correlation Analysis: Income vs. Accessibility

```
cor(accessibility_data$median_income, accessibility_data$nearby_stops, use = "complete.obs")
# Result: -0.0102 (no meaningful relationship)

cor(accessibility_data$median_income, accessibility_data$nearby_groceries, use = "complete.obs")
# Result: +0.2953 (moderate positive relationship)

cor(accessibility_data$median_income, accessibility_data$nearby_schools, use = "complete.obs")
# Result: -0.0996 (weak negative relationship)

cor(accessibility_data$median_income, accessibility_data$nearby_hospitals, use = "complete.obs")
# Result: +0.0042 (no meaningful relationship)
```

Explanation: `cor(x, y)` calculates a number between -1 and 1 describing how strongly two variables move together. Near 0 = no relationship; near 1 = strong positive (both rise together); near -1 = strong negative (one rises as the other falls). `use = "complete.obs"` tells R to skip rows with missing data (the 14 zones with suppressed Census income values) rather than error out.

Finding: grocery access is the only accessibility category with a real income relationship in Chicago — wealthier neighborhoods have measurably better access to supermarkets/groceries, consistent with the well-documented US “food desert” phenomenon. Transit, schools, and hospitals show no meaningful income relationship.

5.4 Visualizing the Relationship

```
library(ggplot2)

ggplot(accessibility_data, aes(x = median_income, y = nearby_groceries)) +
  geom_point(alpha = 0.5, color = "darkgreen") +
  geom_smooth(method = "lm", color = "red") +
  labs(
    title = "Median Income vs. Nearby Grocery Access in Chicago",
    x = "Median Household Income ($)",
    y = "Number of Nearby Grocery Stores"
  )

ggsave("income_vs_groceries.png", width = 8, height = 5, dpi = 300)
```

Explanation of ggplot2’s layered syntax (“grammar of graphics”):

- `ggplot(data, aes(x = ..., y = ...))` — sets the foundation: which table, and which columns map to which axis. `aes()` = “aesthetics.”
- `+ geom_point(...)` — adds a scatter-plot layer, one dot per neighborhood. `alpha = 0.5` makes dots semi-transparent so overlapping density is visible.
- `+ geom_smooth(method = "lm", ...)` — adds a straight best-fit trend line (`"lm"` = linear model) on top, visually confirming the correlation value.
- `+ labs(...)` — adds a title and axis labels.
- `ggsave(...)` — saves the most recently displayed plot as a high-resolution image file.

5.5 Summary Bar Chart of All Four Correlations

```
income_correlations <- data.frame(
  category = c("Transit Stops", "Schools", "Hospitals", "Groceries"),
  correlation = c(
    cor(accessibility_data$median_income, accessibility_data$nearby_stops, use = "complete.obs"),
    cor(accessibility_data$median_income, accessibility_data$nearby_schools, use = "complete.obs"),
    cor(accessibility_data$median_income, accessibility_data$nearby_hospitals, use = "complete.obs"),
    cor(accessibility_data$median_income, accessibility_data$nearby_groceries, use = "complete.obs")
  )
)

ggplot(income_correlations, aes(x = category, y = correlation, fill = category)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  labs(
    title = "Correlation Between Median Income and Accessibility Metrics",
    x = "", y = "Correlation with Median Income"
  ) +
  theme(legend.position = "none")

ggsave("income_accessibility_correlations.png", width = 8, height = 5, dpi = 300)
```

Explanation: `data.frame(...)` builds a small new table from scratch (not from the database) holding the four category names and their correlation values. `geom_col()` draws bars (rather than `geom_point`'s dots). `geom_hline(yintercept = 0, ...)` draws a dashed reference line at zero so positive vs. negative bars are visually obvious at a glance.

5.6 Defining “Underserved” (The ML Target Variable)

```
# Combine the four accessibility counts into one total score per zone
accessibility_data$total_access_score <- accessibility_data$nearby_stops +
  accessibility_data$nearby_schools +
  accessibility_data$nearby_hospitals +
  accessibility_data$nearby_groceries

# Label the bottom 25% of zones (by total score) as "underserved"
threshold <- quantile(accessibility_data$total_access_score, 0.25, na.rm = TRUE)
# threshold = 59

accessibility_data$underserved <- ifelse(accessibility_data$total_access_score <= threshold, 1, 0)

table(accessibility_data$underserved)
# 0: 648 zones (not underserved) 1: 222 zones (underserved, ~25.5%)
```

Explanation: `total_access_score` is a simple sum of the four accessibility counts per zone — a single, easy-to-explain combined measure. `quantile(..., 0.25)` finds the exact value below which the bottom 25% of zones fall (59). `ifelse(condition, 1, 0)` labels each zone: 1 if its score is at or below the threshold (underserved), 0 otherwise. This new `underserved` column becomes the target variable — the thing the machine learning model tries to predict.

5.7 Feature Engineering: Density and Distance from Downtown

Two additional variables were pulled from SQL (queries in Section 4.6) and merged in:

```
area_query <- "SELECT zone_id, ST_Area(geom::geography) AS area_sqm FROM chicago_zones"
area_data <- dbGetQuery(con, area_query)

downtown_query <- "
SELECT
  zone_id,
  ST_Distance(
    geom::geography,
    ST_SetSRID(ST_MakePoint(-87.6298, 41.8781), 4326)::geography
  ) AS distance_from_downtown
FROM chicago_zones
"
downtown_data <- dbGetQuery(con, downtown_query)

accessibility_data <- merge(accessibility_data, area_data, by = "zone_id")
accessibility_data <- merge(accessibility_data, downtown_data, by = "zone_id")

accessibility_data$density <- accessibility_data$population / accessibility_data$area_sqm
```

Explanation: `merge(table1, table2, by = "zone_id")` is R's version of a SQL `JOIN` — combining two tables wherever `zone_id` matches. `density` is simply population divided by area — people per square meter, a measure of how tightly packed a neighborhood is, distinct from raw population count.

A real bug encountered here: merging `downtown_data` in twice across separate script runs created duplicate columns `distance_from_downtown.x` and `distance_from_downtown.y` (R's automatic way of avoiding a silent overwrite when a merge produces a naming collision). Fixed by confirming both were identical (`identical(col.x, col.y)` returned `TRUE`) and keeping just one:

```
accessibility_data$distance_from_downtown <- accessibility_data$distance_from_downtown.x
accessibility_data$distance_from_downtown.x <- NULL
accessibility_data$distance_from_downtown.y <- NULL
```

`column <- NULL` is how a column is deleted in R.

5.8 Model 1: Population + Income Only

```
model <- glm(underserved ~ population + median_income,
  data = accessibility_data,
  family = "binomial")

summary(model)
```

Explanation:

- `glm(...)` = “generalized linear model,” R’s function for this kind of predictive model.
- `underserved ~ population + median_income` — R’s formula syntax: `~` means “predict this using these.” Reads as “predict `underserved` using `population` and `median_income`.”
- `family = "binomial"` — specifies the outcome is a yes/no (0/1) variable, which makes this **logistic regression** (suited to binary outcomes), as opposed to ordinary linear regression (which predicts continuous numbers).

Result: `population` was weakly significant ($p = 0.0455$); `median_income` was not significant ($p = 0.528$). The **p-value** tests whether a variable’s apparent effect is a real pattern or just random noise — a common convention is $p < 0.05 =$ “statistically significant.”

Testing this model’s predictions:

```
accessibility_data$predicted_prob <- predict(model, type = "response")
accessibility_data$predicted_class <- ifelse(accessibility_data$predicted_prob >= 0.5, 1, 0)
table(Actual = accessibility_data$underserved, Predicted = accessibility_data$predicted_class)
```

Result: the model never once predicted “underserved” for any zone — it defaulted to always predicting the majority class (not underserved), technically achieving ~74% “accuracy” while being functionally useless. **Key lesson:** accuracy alone can be deeply misleading when one class dominates the data; a model that always guesses the majority class can look accurate while learning nothing.

5.9 Model 2: Adding Density (Limited Improvement)

```
model2 <- glm(underserved ~ population + median_income + density,
  data = accessibility_data,
  family = "binomial",
  na.action = na.exclude)

summary(model2)
```

New piece: `na.action = na.exclude` — when some rows have missing income data, R’s default behavior silently drops them and re-numbers the results, which misaligns predictions with the original data later. `na.exclude` instead remembers each dropped row’s original position, so any later calculation lines up correctly with gaps (`NA`) left exactly where they belong.

Result: `density` was not significant ($p = 0.366$); AIC (a model-quality score, lower is better) was 978.48, very slightly worse than the first model — no real improvement.

5.10 Model 3: Adding Distance from Downtown (Breakthrough)

```
model3 <- glm(underserved ~ population + median_income + density + distance_from_downtown,
  data = accessibility_data,
  family = "binomial",
  na.action = na.exclude)

summary(model3)
```

Result — all four predictors became statistically significant ($p < 0.001$ for three of four):

Variable	Effect direction	Significance
population	negative (more people → less likely underserved)	$p < 0.001$
median_income	positive (once controlling for distance, higher income → slightly more likely underserved)	$p = 0.00031$
density	positive (denser → more likely underserved)	$p < 0.001$

distance_from_downtown	positive, strongest effect (further from downtown → much more likely underserved)	p < 0.001
------------------------	---	-----------

AIC dropped from 978.48 to **780.46** — a large, genuine improvement in model fit.

Why income's effect flipped direction — confounding: in Model 1, income alone showed no effect. Once distance from downtown is controlled for, a different pattern emerges: among zones at a *similar* distance from downtown, higher-income zones tend to be *slightly* more likely to be underserved — plausibly because some wealthier, lower-density, more car-dependent areas have fewer nearby walkable amenities. This is a real statistical phenomenon (a variable's apparent effect changing once another related variable is added) rather than a mistake.

Main conclusion: distance from downtown is by far the dominant factor — consistent with how Chicago actually developed historically, with transit lines, schools, hospitals, and commercial development concentrated around the city center and along established corridors.

Testing Model 3's predictions (0.5 threshold):

```
accessibility_data$predicted_prob3 <- predict(model3, type = "response")
accessibility_data$predicted_class3 <- ifelse(accessibility_data$predicted_prob3 >= 0.5, 1, 0)
table(Actual = accessibility_data$underserved, Predicted = accessibility_data$predicted_class3)
```

Result: 677/856 correct (~79% accuracy), but only 82 of 220 truly underserved zones correctly caught — **37% recall**.

Improving recall by lowering the classification threshold to 0.35:

```
accessibility_data$predicted_class3_v2 <- ifelse(accessibility_data$predicted_prob3 >= 0.35, 1, 0)
table(Actual = accessibility_data$underserved, Predicted = accessibility_data$predicted_class3_v2)
```

Result: recall improved to **59%** (130 of 220 true underserved zones caught), at the cost of more false positives (88 vs. 41 previously). This is a standard precision/recall trade-off: lowering the decision threshold catches more true cases but also raises more false alarms.

Final save:

```
saveRDS(accessibility_data, "accessibility_data.rds")
```

6. Deployment: Cloud Database + Live Dashboard

6.1 Migrating the Database to Supabase (Cloud PostgreSQL + PostGIS)

A Supabase project was created (free tier), with PostGIS enabled via:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Exporting the local database:

```
$env:PGPASSWORD = "postgres"
& "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" -h localhost -p 5433 -U postgres -d chicago_transit -F c -f
chicago_backup.dump
```

`pg_dump` exports an entire database to a single backup file. `-F c` uses the compressed “custom format.” `$env:PGPASSWORD` sets the password as a temporary environment variable so the tool doesn't need an interactive password prompt.

Importing into Supabase (using the IPv4-compatible “Session pooler” connection, since Supabase's direct connection is IPv6-only):

```
$env:PGPASSWORD = "<supabase-password>"
& "C:\Program Files\PostgreSQL\18\bin\pg_restore.exe" -h aws-0-eu-central-1.pooler.supabase.com -p 5432 -U
postgres.ohcozglrsqilmdogwhyc -d postgres --no-owner --no-privileges -v chicago_backup.dump
```

`--no-owner --no-privileges` skips restoring the original local database's ownership/permission settings, which would conflict with how Supabase manages its own permissions.

Result: all four tables, the `chicago_boundary` table, the `chicago_zones` view, and every spatial index restored successfully to the cloud, confirmed with matching row counts (11,177 / 77,448 / 1,331 / 1,358 / 870).

6.2 The Streamlit Dashboard (`app.py`)

```
import streamlit as st
import psycpg2
import pandas as pd
import folium
from streamlit_folium import st_folium

st.title("Chicago Accessibility Dashboard")
st.write("Exploring transit, school, hospital, and grocery access across Chicago neighborhoods.")

conn = psycpg2.connect(
    dbname=st.secrets["DB_NAME"],
    user=st.secrets["DB_USER"],
    password=st.secrets["DB_PASSWORD"],
    host=st.secrets["DB_HOST"],
    port=st.secrets["DB_PORT"]
)

query = """
SELECT zone_id, tract_name, population, median_income,
       ST_X(ST_Centroid(geom)) AS lon, ST_Y(ST_Centroid(geom)) AS lat
FROM chicago_zones
LIMIT 50
"""
data = pd.read_sql(query, conn)

st.write(f"Showing {len(data)} Chicago neighborhoods")
st.dataframe(data)

st.subheader("Neighborhood Map")
m = folium.Map(location=[41.8781, -87.6298], zoom_start=11)

for _, row in data.iterrows():
    folium.CircleMarker(
        location=[row['lat'], row['lon']],
        radius=5,
        popup=f"{row['tract_name']}: {row['population']} people",
        color="blue",
        fill=True
    ).add_to(m)

st_folium(m, width=700, height=500)
```

Explanation:

- `st.title(...)`, `st.write(...)`, `st.dataframe(...)` — Streamlit functions that turn a plain Python script into a web app with almost no web-development code required.
- `st.secrets["DB_NAME"]` etc. — reads database credentials from a secrets file (`.streamlit/secrets.toml` locally, or entered directly in Streamlit Cloud's dashboard for the deployed version) rather than hardcoding them in the script. This keeps the password out of the public GitHub repository. **Security lesson:** any code destined for a public repo must never contain real credentials directly — a secrets/environment-variable approach is standard practice.
- `ST_X(ST_Centroid(geom))` / `ST_Y(...)` — new SQL functions used here: since each zone needs to be plotted as a single point on the map, `ST_Centroid(geom)` finds the shape's center point (same centroid concept as the POI conversion in Section 3.4), and `ST_X` / `ST_Y` extract that point's longitude and latitude as plain numbers.
- `folium.Map(location=[41.8781, -87.6298], zoom_start=11)` — creates an interactive map centered on downtown Chicago.
- The `for` loop adds one clickable circle marker per neighborhood, using the same row-iteration pattern seen in `load_stops.py`.
- `st_folium(m, ...)` — the connector that renders the folium map inside the Streamlit app.

A real bug encountered: the initial deployment failed because the local, hardcoded `localhost / chicago_transit` connection details had never actually been replaced with the `st.secrets` version before pushing to GitHub — the edit existed only in the plan, not in the saved file. Lesson: always verify the actual file content matches what you intend to have changed, especially before pushing or deploying.

6.3 requirements.txt (tells Streamlit Cloud which packages to install)


```
streamlit
psycpg2-binary
pandas
folium
streamlit-folium
```

6.4 Secrets File (`.streamlit/secrets.toml` , never committed to GitHub)

```
DB_HOST = "aws-0-eu-central-1.pooler.supabase.com"
DB_PORT = "5432"
DB_NAME = "postgres"
DB_USER = "postgres.ohcozglrsqilmdogwhyc"
DB_PASSWORD = "<your-password>"
```

Added to `.gitignore` so it is never pushed to the public repository. The same values are entered directly into Streamlit Cloud's "Secrets" panel when deploying, so the live app can authenticate without the password ever appearing in code.

6.5 Final GitHub Repository Structure

```
chicago-accessibility-analysis/
├── load_stops.py
├── load_roads.py
├── load_population.py
├── load_poi.py
├── get_chicago_boundary.py
├── queries.sql
├── analysis.R
├── analysis.Rmd / analysis.html
├── app.py
├── requirements.txt
├── README.md
├── income_vs_groceries.png
├── income_accessibility_correlations.png
└── .gitignore
```

Live links: - GitHub: <https://github.com/Abiodun360of/chicago-accessibility-analysis> - Live dashboard: <https://chicago-accessibility-analysis-idfqtytqpujbletwrmbxxh.streamlit.app/> ## 7. How to Resume Work in RStudio (Practical Restart Guide)

Every time RStudio is closed and reopened, all variables in memory — `con` , `query` , `accessibility_data` , `model3` , everything — disappear. This is normal and expected, not a bug. The fix is to always re-run a short "startup block" at the beginning of each new session before continuing any analysis.

The Startup Block (run this first, every session)

```
library(DBI)
library(RPostgres)

con <- dbConnect(
  RPostgres::Postgres(),
  dbname = "chicago_transit",
  host = "localhost",
  port = 5433,
  user = "postgres",
  password = "postgres"
)

# Confirm the connection works before doing anything else
dbGetQuery(con, "SELECT 1")

# Reload the saved dataset instantly (no need to re-run the slow 30-40 second query)
accessibility_data <- readRDS("accessibility_data.rds")
nrow(accessibility_data) # should show 870

# Confirm all the extra columns built during the ML phase are present
colnames(accessibility_data)
# Should include: zone_id, tract_name, population, median_income, nearby_stops,
# nearby_schools, nearby_hospitals, nearby_groceries, total_access_score,
# underserved, area_sqm, density, distance_from_downtown
```

If `accessibility_data.rds` is missing the ML columns (an older save from before Section 5.6–5.7), rebuild them with:

```

accessibility_data$total_access_score <- accessibility_data$nearby_stops +
  accessibility_data$nearby_schools +
  accessibility_data$nearby_hospitals +
  accessibility_data$nearby_groceries

threshold <- quantile(accessibility_data$total_access_score, 0.25, na.rm = TRUE)
accessibility_data$underserved <- ifelse(accessibility_data$total_access_score <= threshold, 1, 0)

area_data <- dbGetQuery(con, "SELECT zone_id, ST_Area(geom::geography) AS area_sqm FROM chicago_zones")
downtown_data <- dbGetQuery(con, "
  SELECT zone_id,
    ST_Distance(geom::geography, ST_SetSRID(ST_MakePoint(-87.6298, 41.8781), 4326)::geography) AS
distance_from_downtown
  FROM chicago_zones
")

accessibility_data <- merge(accessibility_data, area_data, by = "zone_id")
accessibility_data <- merge(accessibility_data, downtown_data, by = "zone_id")
accessibility_data$density <- accessibility_data$population / accessibility_data$area_sqm

# Save the rebuilt version so this doesn't need repeating again
saveRDS(accessibility_data, "accessibility_data.rds")

```

Rebuilding the Final Model (fast, no database query needed)

```

model3 <- glm(underserved ~ population + median_income + density + distance_from_downtown,
  data = accessibility_data,
  family = "binomial",
  na.action = na.exclude)

summary(model3)

```

Common Errors and Fixes Encountered (for quick reference)

Error message	Cause	Fix
could not find function "dbGetQuery"	library(DBI) not loaded in this session	Run library(DBI) again
object 'con' not found	Session was restarted/interrupted	Re-run the full startup block above
replacement has X rows, data has Y rows	Model dropped rows with missing data without remembering their positions	Add na.action = na.exclude when building the model
Query appears stuck for 10+ minutes	A previous interrupted query left a lock, or a genuine slow double-join	Check pg_stat_activity in pgAdmin; if truly stuck, use pg_terminate_backend(pid)
Duplicate columns like x.x / x.y after merge()	Same data merged in twice	Confirm both are identical with identical(a, b), keep one, delete the other with col <- NULL
find_srid() error during .to_postgis()	Geopandas' default geometry column name (geometry) doesn't match the database table's column name (geom)	Use .rename_geometry('geom') before writing

8. Final Summary of Key Findings

- Grocery/food access — not transit or healthcare — shows the clearest income disparity in Chicago.**
Correlation between median income and nearby grocery stores: +0.295 (moderate). Transit stops, schools, and hospitals all showed correlations near zero, meaning income is not a strong driver of those types of access on its own.
- Distance from downtown is the single strongest predictor of poor accessibility** — stronger than income, population, or density. This is consistent with how Chicago's transit and commercial infrastructure historically developed, concentrated around the city center and along established corridors.
- A logistic regression model combining population, income, density, and distance from downtown achieved statistically significant results for all four variables** (AIC improved from 978 to 780 compared to simpler models), correctly identifying 59% of genuinely underserved neighborhoods when using a 0.35 classification threshold — a real, usable, honestly-evaluated result with a clearly reported limitation (41% of true cases still missed).

4. **Basic demographics alone only partially explain accessibility gaps**, suggesting that historical infrastructure planning and geography play a larger role than current population/income patterns — a mature, evidence-based conclusion rather than an overstated claim.

Appendix: Full Project File Reference

File	Purpose
load_stops.py	Loads CTA transit stop data into PostGIS
load_roads.py	Loads Chicago road network from OpenStreetMap
load_population.py	Loads Census tract boundaries + population/income data
load_poi.py	Loads schools, hospitals, groceries from OpenStreetMap
get_chicago_boundary.py	Loads Chicago’s official city boundary (bug fix)
queries.sql	All SQL table creation, indexing, and analysis queries
analysis.R	Full R script: database connection, statistics, ML model
analysis.Rmd / analysis.html	Knitted report combining findings and charts
app.py	Streamlit live dashboard (deployed)
requirements.txt	Python package list for deployment
README.md	Project overview for GitHub
accessibility_data.rds	Saved R dataset (not on GitHub — regenerable via the scripts above)

End of document.